DATE

PAGE

Generative AI

Notes

- Introduction to Generative AI
- How Generative AI Work's
- Popular Models and Architectures
- Prompting Techniques
- Applications
- Tools and Platforms
- Ethics and Limitations

Name	:	Zaka ur Rehman
Subject	:	Generative AI
Pages	:	20

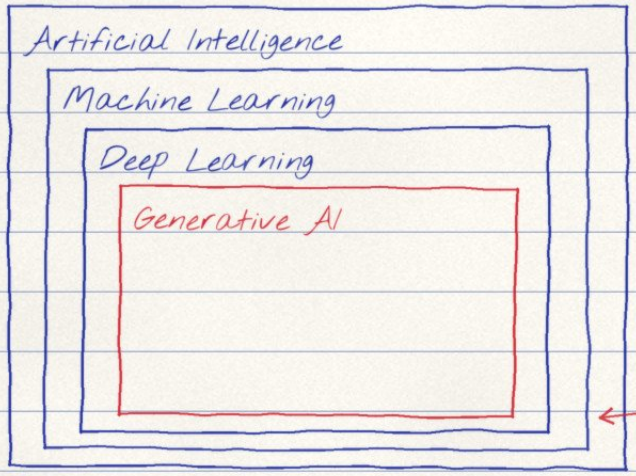
1. Introduction to Generative AI

What is Generative AI?

☆ Generative AI is a branch of artificial intelligence that can create new content like text, images, audio, video and code by learning patterns from existing data.

- Does not only analyse data - it generates brand new output
- Learns patterns, structure and style from huge datasets
- Works on a prompt given by the user (text, image or voice)
- Examples: ChatGPT, Claude, Gemini, DALL-E, Midjourney

AI vs ML vs DL vs GenAI



AI - machines that copy human intelligence

ML - learns patterns from data, no fixed rules

DL - ML with deep neural networks

GenAI - DL models that create new content

Discriminative vs Generative Models

	Discriminative	Generative
Goal	classify or predict a label	create new data
Learns	boundary between classes P(y|x)	distribution of the data P(x)
Examples	spam filter, image classifier	ChatGPT, DALL-E, Suno
Output	label or number	text, image, audio, video

Evolution of Generative AI

- 1966 ELIZA - first chatbot (rule based, MIT)
- 2013 VAE (Variational Autoencoder) introduced
- 2014 GANs by Ian Goodfellow - realistic images
- 2017 Transformer - "Attention Is All You Need"
- 2018 GPT-1 (OpenAI) and BERT (Google)
- 2020 GPT-3 - 175 billion parameters
- 2021 DALL-E - text to image
- 2022 Stable Diffusion + ChatGPT goes viral
- 2023 GPT-4, Claude, Llama, Gemini - LLM boom
- 2024+ Multimodal models, AI agents, video (Sora)

Types of Content Generated

→ Text - essays, emails, summaries, chat answers

☆ → Images - art, logos, product photos, designs

→ Audio - voice cloning, music, text to speech

→ Video - clips from a text prompt, animation

→ Code - functions, bug fixes, full apps

→ Synthetic data and 3D models

Why is it booming now?

✓ Huge amounts of data on the internet

✓ Powerful hardware - GPUs and TPUs

✓ Transformer architecture scales very well

✓ Easy access through chat apps, APIs, open-source models

Key Terms to Remember

- Prompt: the input or instruction we give to the model
- Token: small piece of text (word or part of a word) that the model reads. 1 token is about 4 characters in English
- Parameters: the learned numbers (weights) inside the model. More parameters = more capacity (7B, 70B ...)
- Embedding: a list of numbers (vector) that captures the meaning of a token
- LLM: Large Language Model - trained on huge text data to understand and generate language
- Foundation model: big general model trained once, then adapted for many tasks
- Context window: maximum amount of text (tokens) the model can look at at one time
- Inference: using a trained model to produce output (not training)
- Fine-tuning: extra training on a small, specific dataset to specialise the model
- Temperature controls randomness. Low = focused, high = creative
- Hallucination: model confidently gives wrong or made-up information

> Remember: A language model does not "know" facts like a database. It predicts the most likely next token from patterns it has learned.

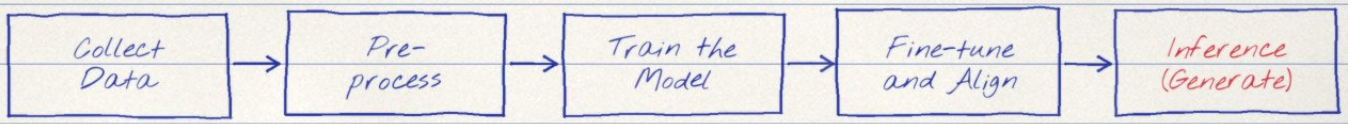
Common Abbreviations

- GPT - Generative Pre-trained Transformer. NLP - Natural Language Processing
- GAN - Generative Adversarial Network. VAE - Variational Autoencoder
- RAG - Retrieval Augmented Generation. SFT - Supervised Fine-Tuning
- RLHF - Reinforcement Learning from Human Feedback

2. How Generative AI Works

Basic idea: the model learns the probability distribution of the training data, then samples from it to produce new data.

Overall Pipeline



→ Data - books, websites, code, images, papers (billions of examples)
→ Pre-processing - clean, remove duplicates, tokenize
→ Training - model adjusts its weights to reduce prediction error
→ Fine-tuning - learn to follow instructions, be safe and helpful
→ Inference - user gives prompt, model generates output token by token

Example: Next Word Prediction

Prompt: "The sky is ____"
- blue → 62% (picked most of the time)
- clear → 15%
- falling → 1%

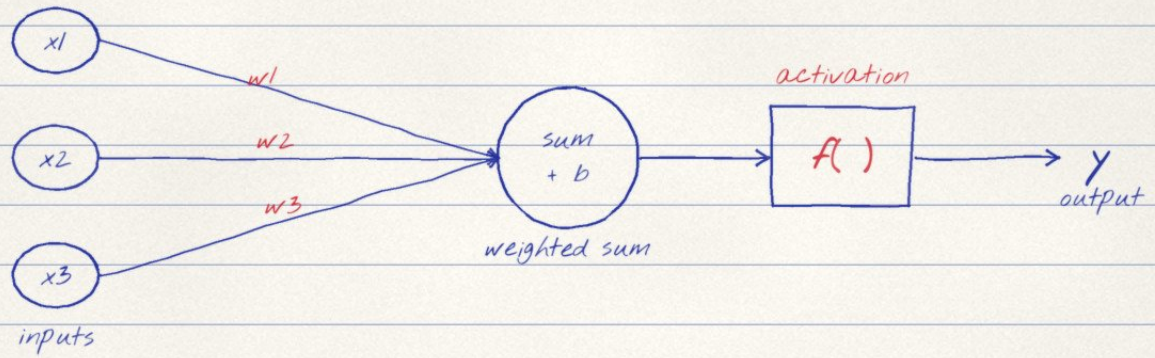
Generation loop: predict next token → add it to the text → predict again ... until the answer is complete

Common Generation Tasks

→ Text to text - chat, translation, summary
→ Text to image - DALL-E, Midjourney, Stable Diffusion
→ Image to text - captions, reading charts and documents
→ Text to speech or video - voice-overs, short clips

Neural Network Basics

Generative AI models are built using deep neural networks - layers of connected "neurons" inspired by the human brain.



$$y = f(w1.x1 + w2.x2 + w3.x3 + b)$$

- Weights (w) - strength of each connection, learned in training
- Bias (b) - extra number that shifts the output
- Activation - ReLU, sigmoid, softmax, adds non-linearity
- Layers - input layer, hidden layers, output layer
- Deep learning = network with many hidden layers

How a Network Learns

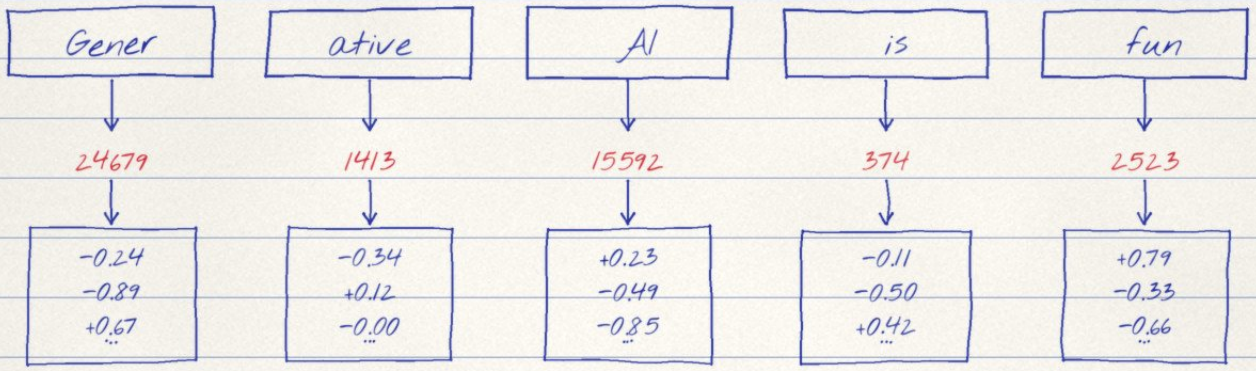
→ Forward pass → prediction → loss measures the error
→ Backpropagation finds how much each weight caused the error
→ Gradient descent updates weights a little (learning rate)
→ Repeat for millions of steps (epochs) until loss is low

Why GPUs? Training needs huge matrix multiplications. GPUs and TPUs do thousands of them in parallel, so training is much faster than on a normal CPU.

Tokenization and Embeddings

Models cannot read words directly. Text is converted: text → tokens → token IDs → embeddings (vectors)

Text: "Generative AI is fun"



Tokenization

- Splits text into tokens using BPE, WordPiece or SentencePiece
- ☆ Rule of thumb: 1 token is about 4 characters or 0.75 word
- Pricing and context limits are counted in tokens

Embeddings

- Each token → vector of hundreds or thousands of numbers
- Similar meanings are close together in vector space
- Famous example: King - man + woman ≈ queen
- Similarity measured by cosine similarity
- Positional encoding is added so the model knows word order

Example

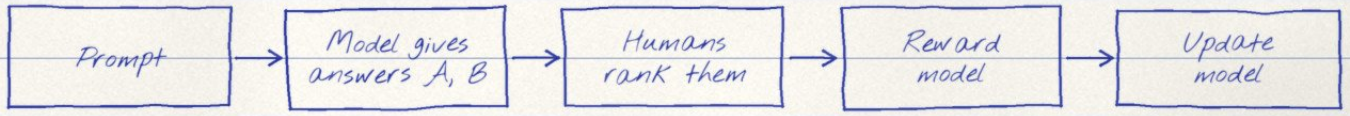
- "unbelievable" → un + believ + able (3 tokens)

Context window = tokens in the prompt + tokens in the answer. Longer window = model can read bigger documents.

Stages of Training an LLM

- 1. Pre-training - model reads trillions of tokens and learns to predict the next token. Self-supervised, needs huge compute (thousands of GPUs, weeks or months).
- 2. Supervised Fine-tuning (SFT) - trained on example prompts with good answers written by humans, so it follows instructions.
- 3. RLHF and Alignment - Reinforcement Learning from Human Feedback makes answers more helpful, honest and safe.

RLHF Steps



Training vs Inference

	Training	Inference
What	learn weights from data	use fixed weights to answer
Cost	very high (weeks, GPUs)	lower per request
Done	once (or occasionally)	every time user prompts

Scaling Laws

More data + more parameters + more compute → usually better performance. But gains slow down and cost rises fast.

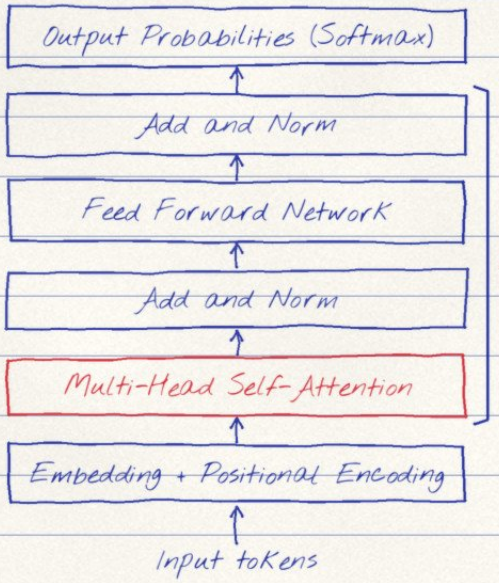
Fine-tuning Methods

- Full fine-tuning - update all weights (costly)
- LoRA - train small adapter matrices only, cheap and fast
- Instruction tuning - teach the model to follow commands

3. Popular Models and Architectures

Transformer Architecture

- Introduced by Google in 2017 (paper: Attention Is All You Need)
- Replaced RNN and LSTM - processes all tokens in parallel
- Backbone of GPT, Claude, Gemini, Llama, BERT, T5



Self-attention lets every token look at all other tokens and decide which ones matter.

Each token makes 3 vectors:

Q (Query) - what am I looking for?

K (Key) - what do I contain?

V (Value) - the information to pass on

Scores = Q.K → softmax → weighted sum of V

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{dK}}\right) \cdot V$$

- Multi-head attention - several attention heads learn different relations
- Residual connections + layer norm keep training stable

Why Transformers Won

✓ Parallel training on GPUs - much faster than RNN

✓ Captures long-range relations between words

✓ Scales well with more data and parameters

Types of Transformer Models

Type	Examples	Best for
Encoder-only	BERT, RoBERTa	understanding text: classification, search, NER
Decoder-only	GPT, Claude, Llama, Gemini	text generation, chat, code (autoregressive)
Encoder-Decoder	T5, BART	translation, summarisation

Large Language Models (LLMs)

- Decoder-only transformers with billions of parameters
- Autoregressive: generate one token at a time, each new token depends on previous ones
- Emergent abilities: reasoning, translation, coding appear at scale
- Small Language Models (SLMs) run on laptops and phones

Decoding and Sampling Settings

Setting	What it does
Greedy	always picks the most likely token (safe, repetitive)
Temperature	low (0–0.3) = focused; high (0.8+) = creative
Top-K	choose only from the K most likely tokens
Top-p	choose from smallest set whose probability adds up to p
Max tokens	limit on length of the output

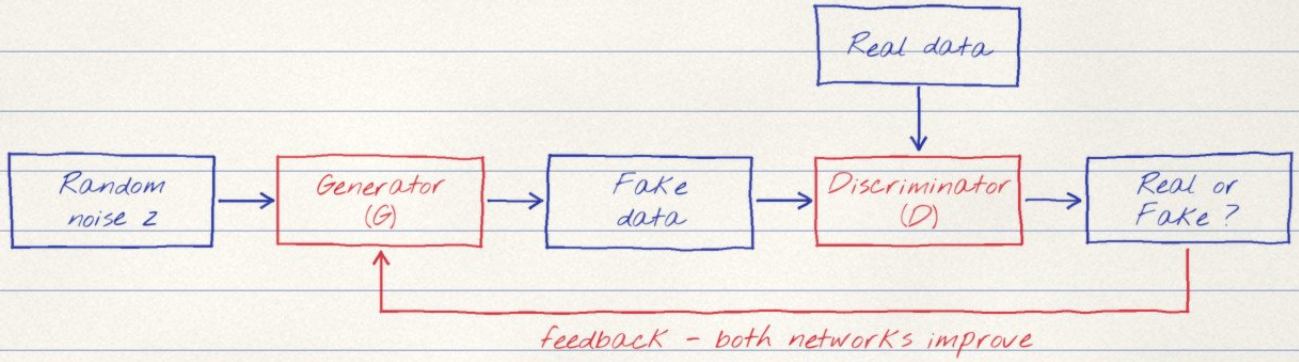
Note: Same prompt can give different answers when temperature > 0.

Context Window

- Number of tokens the model can handle at once (8K, 128K, 1M+)
- Text beyond the window is forgotten or must be shortened

GAN - Generative Adversarial Network

- Proposed by Ian Goodfellow in 2014
- Two networks play a game: Generator makes fake data, Discriminator tries to detect fakes

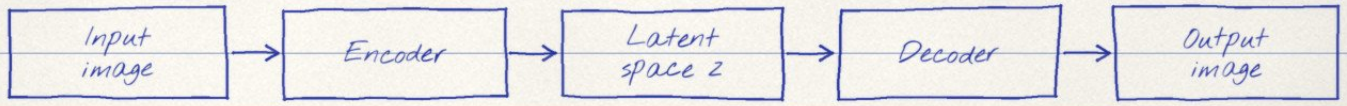


✓ Good: very sharp, realistic images, fast generation

– Problems: mode collapse, unstable training

→ Uses: face generation, style transfer, deepfakes, super-resolution

VAE - Variational Autoencoder



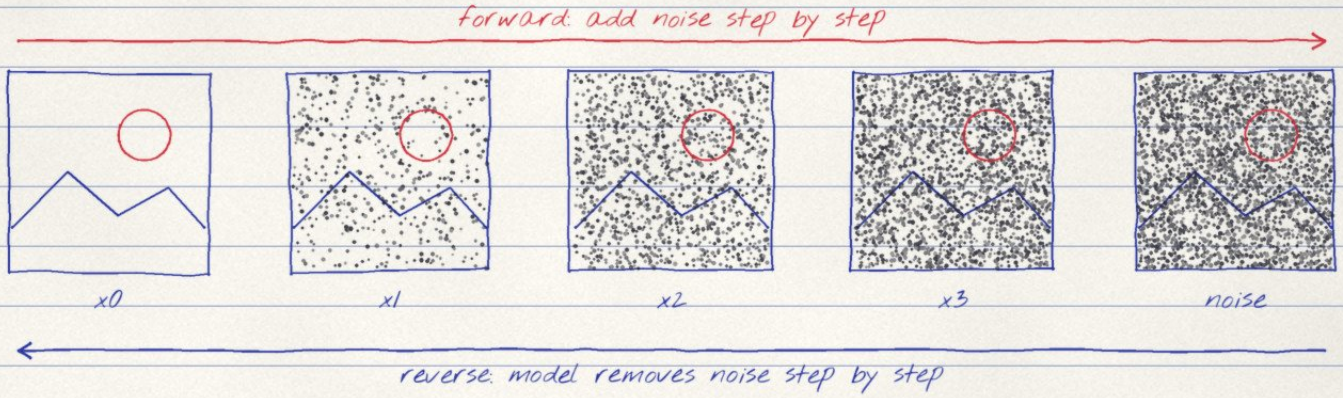
- Encoder compresses data into a smooth latent space; decoder rebuilds it
- Sample a new point in latent space → decode → new data
- More stable than GANs but outputs can be slightly blurry

Other Approaches

- Autoregressive - predict the next piece (GPT, WaveNet)
- Flow-based - exact likelihood, reversible transformations

Diffusion Models

Currently the most popular approach for image and video generation.



- Forward process - slowly add Gaussian noise to a real image until it is pure noise
- Reverse process - a neural network (often U-Net) learns to predict and remove the noise
- To generate: start from random noise → denoise many steps → image
- Text prompt guides denoising through a text encoder (e.g. CLIP)
- Latent diffusion (Stable Diffusion) works in a compressed latent space - cheaper and faster
- Examples: Stable Diffusion, DALL-E, Imagen, Midjourney, Sora (video)

GAN vs Diffusion

	GAN	Diffusion
Speed	fast (one pass)	slower (many steps)
Training	unstable	stable
Quality	good, may collapse	excellent, diverse

Multimodal models can take text + image + audio together (e.g. GPT-4o, Gemini, Claude).

Popular Generative AI Models

Model	Company	Type / Use
GPT series	OpenAI	LLM, multimodal chat, ChatGPT
Claude	Anthropic	LLM, long context, coding, analysis
Gemini	Google DeepMind	multimodal LLM (text, image, audio, video)
Llama	Meta	open-weight LLM family
Mistral	Mistral AI	efficient open-weight LLMs
DeepSeek	DeepSeek	open-weight LLM and reasoning models
DALL-E	OpenAI	text to image
Stable Diffusion	Stability AI	open image generation
Midjourney	Midjourney Inc.	artistic image generation
Sora	OpenAI	text to video
Whisper	OpenAI	speech to text
BERT	Google	encoder model for language understanding

Open-weight vs Closed Models

	Open-weight	Closed (API)
Access	download and run locally	use via app or API
Control	full - fine-tune, private	limited to API options
Examples	Llama, Mistral, DeepSeek	GPT, Claude, Gemini

Model size is written in parameters: 7B, 13B, 70B ... (B = billion).

4. Prompting Techniques

Prompt engineering = writing clear instructions so the model gives the best possible output.

Anatomy of a Good Prompt

- Role - "You are an expert Python teacher"
- Task - what exactly to do
- Context - background, audience, goal
- Format - list, table, JSON, 100 words ...
- Examples - show how the answer should look

Bad vs Good Prompt

- Bad: "Tell me about AI"
✓ Good: "Explain generative AI to a class 10 student in 5 bullet points with one real-life example."

Zero-shot, One-shot, Few-shot

- Zero-shot - only the instruction, no examples

e.g. "Classify sentiment: The movie was great."

- One-shot - one example given before the task
- Few-shot - 2-5 examples so the model learns the pattern

Review: "Loved it" → Positive
Review: "Waste of money" → Negative
Review: "Not bad at all" → ?

Template: Act as [role]. Task: [what]. Audience: [who]. Format: [how]. Limit: [length].

Advanced Prompting Techniques

- ✯ Chain-of-Thought (CoT) - ask the model to "think step by step"; improves maths and logic
- ✯ Role prompting - assign persona: teacher, lawyer, editor
- ✯ Self-consistency - generate many reasoning paths, take the most common answer
- ✯ Prompt chaining - break a big task into small prompts; output of one is input of next
- ✯ ReAct - Reason + Act: model thinks, uses a tool (search, calculator), then continues
- ✯ Tree of Thoughts - explore several reasoning branches and choose the best
- ✯ System prompt - hidden instructions that set behaviour and rules of the assistant

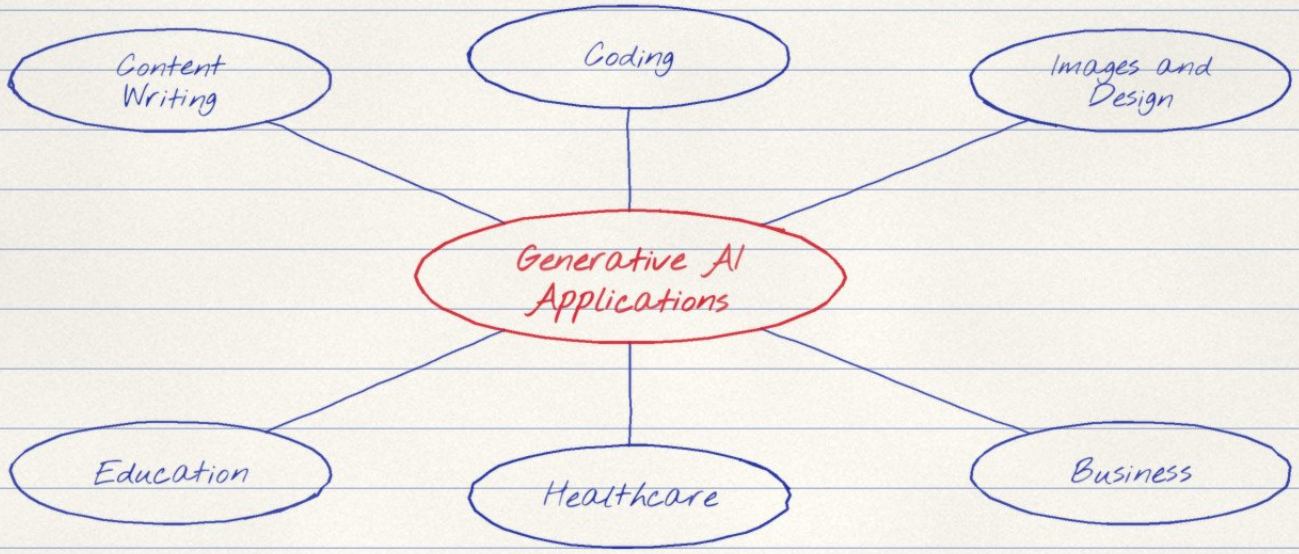
Useful Tips

- ✓ Be specific - give length, tone and audience
- ✓ Give examples of the output you want
- ✓ Ask for a fixed format (table, JSON, bullets)
- ✓ Iterate - refine the prompt after seeing the answer
- ✓ Provide context or paste the source text
- ✓ Always verify important facts yourself

Do and Do Not

Do	Do not
clear, detailed instructions	vague one-line prompts
one task at a time	mix many unrelated tasks
give reference material	expect it to guess private facts
check answers	blindly trust the output

5. Applications



→ Content creation - blogs, ads, scripts, social media posts, translation

→ Software development - code completion, debugging, test cases, documentation

→ Design and media - logos, posters, product mock-ups, music, video, voice-overs

→ Education - personal tutor, quiz and notes generation, language practice

→ Healthcare - clinical note summaries, drug discovery, medical imaging support

→ Business - customer support chatbots, report summaries, market research, HR

→ Science and research - protein design, literature review, data analysis

→ Gaming - characters, stories, worlds and textures made automatically

6. Tools and Platforms

Category	Popular Tools
AI assistants	ChatGPT, Claude, Gemini, Microsoft Copilot, Perplexity
Image	Midjourney, DALL-E, Stable Diffusion, Adobe Firefly, Canva AI
Video	Sora, Runway, PiKa, Veo
Audio and Voice	ElevenLabs, Suno, Whisper
Coding	GitHub Copilot, Cursor, Claude Code, Codex
Frameworks	PyTorch, TensorFlow, Hugging Face Transformers
LLM apps	LangChain, LlamaIndex, OpenAI and Anthropic APIs
Run locally	Ollama, LM Studio, llama.cpp
Vector DB	Pinecone, Chroma, FAISS, Weaviate
Cloud	Google Vertex AI, AWS Bedrock, Azure AI

Hugging Face

- Hub for thousands of open models and datasets
- "transformers" library: load a model in a few lines of Python

Using an LLM API

- Send messages (prompt) + settings → receive generated text
- Billed by input and output tokens
- Keep API keys secret - never put them in public code

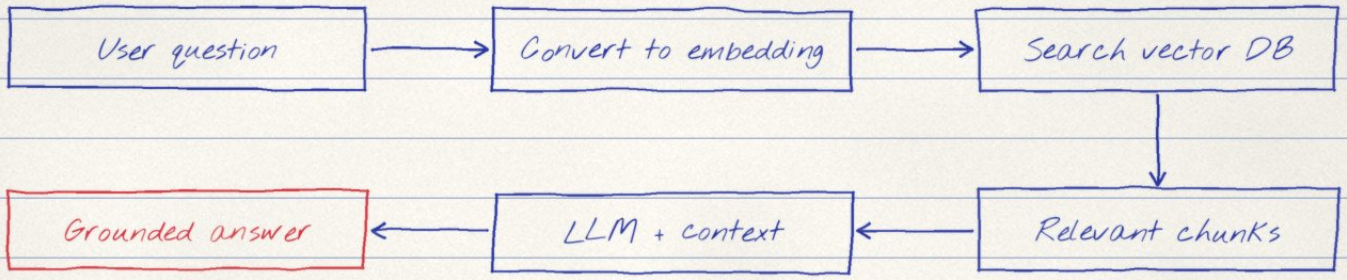
Choosing a Tool

→ Need privacy → run an open-weight model locally

→ Need best quality → top closed model through API

RAG - Retrieval Augmented Generation

Gives the model fresh or private knowledge by retrieving documents and adding them to the prompt.



✓ Reduces hallucination, answers can show sources

✓ No need to retrain the model when data changes

Prompting vs RAG vs Fine-tuning

	Prompting	RAG	Fine-tuning
Effort	lowest	medium	highest
Changes	instructions	knowledge	model behaviour
Use when	quick tasks	company docs, latest facts	fixed style, domain skill

AI Agents

- LLM + tools (search, code, APIs) + memory + planning

→ Loop: think → choose tool → observe result → repeat until goal done

- Examples: coding agents, research agents, customer-support agents
- Need guardrails and human approval for risky actions

Remember: Fine-tuning teaches style and skills, RAG supplies facts.

7. Ethics and Limitations

Main Ethical Concerns

- ☆ Bias and fairness - models copy biases present in training data
- ☆ Hallucination / misinformation - confident but false answers, fake news at scale
- ☆ Deepfakes - fake faces, voices and videos used for fraud or defamation
- ☆ Copyright and ownership - training on copyrighted work; who owns AI output?
- ☆ Privacy and data leaks - personal data in prompts or training data may be exposed
- ☆ Job impact - some tasks automated, new skills and jobs needed
- ☆ Environmental cost - training and running models uses a lot of energy and water
- ☆ Security misuse - phishing, malware help, prompt injection, jailbreaks
- ☆ Over-reliance / cheating - students and workers may stop thinking for themselves

How to Reduce Risks

- ✓ Check important facts from reliable sources
- ✓ Never paste passwords, private or confidential data
- ✓ Label AI generated content, give credit
- ✓ Keep a human in the loop for important decisions
- ✓ Test models for bias and unsafe outputs

Case: In 2023 a lawyer submitted a court filing with fake case citations made up by ChatGPT - always verify!

Technical Limitations

- No real understanding - predicts patterns, can fail at simple logic
- Knowledge cutoff - does not Know events after training (unless it can search)
- Limited context window - may forget the early parts of long inputs
- Output can change between runs; sensitive to wording of the prompt
- High cost of training and running large models
- Black box - hard to explain why a model gave an answer

Responsible AI Principles

Fairness . Transparency . Accountability . Privacy . Safety . Human oversight

- Regulation: EU AI Act (risk-based rules), data protection laws like GDPR

Future Trends

→ Autonomous AI agents that complete multi-step tasks
→ Better multimodal models (text + image + audio + video)
→ Reasoning models and longer context
→ Smaller, faster models running on-device

Key Takeaways

✓ GenAI creates new content by learning patterns from data
✓ Transformers + diffusion models power today's tools
✓ Good prompts = good results; always verify and use ethically